

Фотореалистичный рендеринг (*aka raytracing*)

Лекция 6: Alias method, bidirectional path tracing,
Metropolis light transport, формат glTF

2024

Importance sampling lights

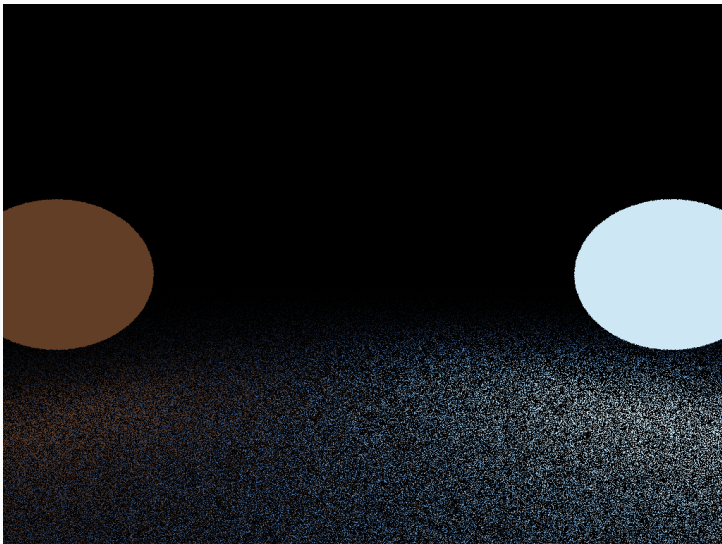
- При importance sampling мы выбирали случайный источник равномерно
- Это не всегда самая лучшая стратегия:
 - Источники отличаются размером
 - Источники отличаются яркостью
- $\implies$ Хочется какую-то эвристику

Weighted lights sampling

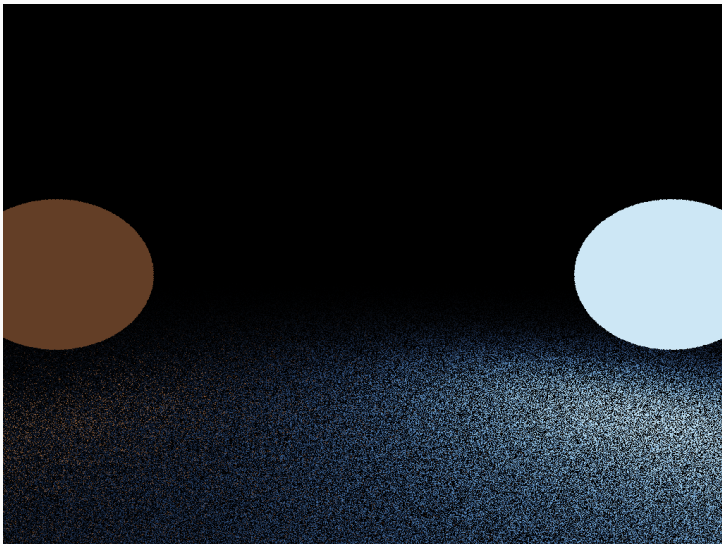
- В качестве эвристического веса w_i источника света можно взять
 - Яркость источника
 - Площадь поверхности источника
 - Их произведение
- Вероятность выбрать конкретный источник равна

$$p_i = \frac{w_i}{\sum w_j}$$

Uniform lights sampling



Weighted lights sampling



Weighted lights sampling

- Для семплинга источников света с учётом такой эвристики, нужно уметь семплить произвольное дискретное распределение, заданное набором вероятностей
- Эта операция выполняется почти на каждый луч $\implies$ хочется делать это быстро

Weighted sampling: постановка задачи

- Есть дискретное распределение с заданными вероятностями исходов p_i для $i \in \{1 \dots N\}$
- Хотим научиться семплить случайную величину X такую, что $\mathbb{P}(X = i) = p_i$

Weighted sampling: линейный алгоритм

- Генерируем случайную величину $U \sim U(0, 1)$
- Если $U \leq p_1$, возвращаем $X = 1$
- Если $p_1 < U \leq p_1 + p_2$, возвращаем $X = 2$
- Если $p_1 + p_2 < U \leq p_1 + p_2 + p_3$, возвращаем $X = 3$
- И т.д.

Weighted sampling: линейный алгоритм

- Есть разные варианты реализации:
 - Можно считать сумму вероятностей всех посещённых индексов в процессе семплинга
 - Можно заранее предподсчитать префиксные суммы $s_k = \sum_{i=1}^k p_i$
 - Можно вычитать вероятности посещённых индексов из U

Weighted sampling: логарифмический алгоритм

- Линейная сложность – очень медленно, если источников света много
- Если заранее предподсчитать префиксные суммы, то нужный индекс можно найти бинарным поиском
- Находим минимальный i такой, что $U \leq s_i$
- В C++ это `std::upper_bound`
- Сложность такого подхода – логарифм от числа источников света

Weighted sampling: alias method

- Логарифм – уже очень хорошо, но не идеально
 - Сложность всё ещё растёт с числом источников света
 - Бинарный поиск плохо дружит с кешами памяти, так как делает много практически случайных обращений к массиву
- Есть метод, работающий за константное время с линейной предобработкой: *alias method*

Alias method (Walker, 1974)

- Рассмотрим особый класс дискретных распределений, заданных двумя массивами $q_i \in [0, 1]$ (*probability table*) и $k_i \in \{1 \dots N\}$ (*alias table*) и алгоритмом семплинга:
 - Генерируем равномерно $i \in \{1 \dots N\}$ и $u \in [0, 1]$
 - Если $u \leq q_i$, возвращаем i
 - Иначе, возвращаем k_i
- Утверждение: **любое** дискретное распределение можно привести к такому виду
- Элемент k_i называется *alias'ом* элемента i

Alias method: доказательство

- 1. Положим $q_i = N \cdot p_i$ и $k_i = i$
- Заметим, что $\sum q_i = N \cdot \sum p_i = N$
- 2. Если все $q_i = 1$, то все $p_i = \frac{1}{N}$, и алгоритм можно завершить
- 3. Иначе, выберем $q_i < 1$ и $q_j > 1$
 - Они существуют из-за того, что $\sum q_i = N$
- 4. 'Откусим' часть вероятности q_j , чтобы дополнить ей q_i :
 - 4.1. Положим $k_i = j$
 - 4.2. Обновим $q_j \leftarrow q_j - (1 - q_i)$
- 5. Удалим i -ый элемент из рассмотрения
- Сумма q_i для оставшихся элементов равна $N - q_i - (1 - q_i) = N - 1$
- 6. $\implies$ Инвариант на сумму q_i сохранился, возвращаемся к пункту 2

Alias method: алгоритм

- Приведённое доказательство конструктивно, и уже является алгоритмом построения массивов q_i и k_i
- Однако, на каждом шаге нам нужно за линейное время искать элементы $q_i < 1$ и $q_j > 1 \implies$ квадратичная сложность подсчёта
- К тому же, алгоритм страдает от ошибок округления: инвариант $\sum q_i = N$ может нарушиться, и один из элементов q_i, q_j может не найтись
- Вместо линейного поиска можно хранить две очереди с элементами меньше и больше 1
- Можно заменить вычисление $q_j - (1 - q_i)$ численно более стабильным $(q_j + q_i) - 1$
- $\implies$ *Vose alias method*

Vose alias method (Vose, 1991)

- 1. Положим $q_i = N \cdot p_i$
- 2. Заведём две очереди:
 - 2.1. **small** с индексами, для которых $q_i < 1$
 - 2.2. **large** с индексами, для которых $q_i \geq 1$
- 3. Пока обе очереди не пусты:
 - 3.1. Достанем элементы $i \in \text{small}$ и $j \in \text{large}$
 - 3.2. Положим $k_i = j$
 - 3.3. Обновим $q_j \leftarrow (q_i + q_j) - 1$
 - 3.4. Если $q_j < 1$, положим j в **small**, иначе – в **large**
- 4. Если одна из очередей опустела, то для всех элементов второй очереди положим $q_i = 1$ и $k_i = i$

Alias method: визуальная интерпретация

- У этого метода есть забавная визуальная интерпретация в терминах разрезания и упаковки прямоугольников



Alias method

- Alias-таблица для конкретного распределения *не уникальна*
- Сэмплинг работает чуть быстрее, если пытаться максимизировать значения q_i (будет меньше обращений к массиву k_i)
- Построение оптимальных массивов q_i и k_i – **NP-трудная** задача
- Можно использовать жадный алгоритм, всегда выбирая $i \in \text{small}$ с наибольшим q_i и $j \in \text{large}$ с наименьшим q_j – для этого нужны приоритетные очереди, и время предобработки вырастает до $O(N \cdot \log N)$
- Очень подробная статья с выводом алгоритма:
keithschwarz.com/darts-dice-coins

Unidirectional path tracing

- Алгоритм, который мы используем, называется *unidirectional path tracing*: мы генерируем случайный путь света в *одном направлении*, т.е. из камеры в сцену
- Многие индустриальные рендереры используют этот алгоритм (напр. движок Cycles в Blender'e)

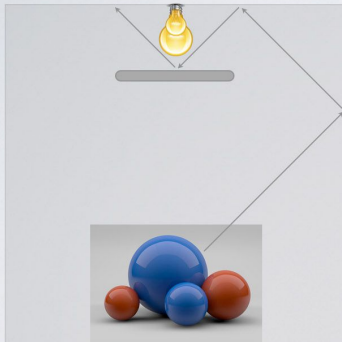
Unidirectional path tracing

- Ресар: генерировать путь из источника света – плохая идея: он вряд ли (с вероятностью 0) попадёт в камеру
- Ресар: генерировать путь из камеры – плохая идея: он может не попасть в источник света
- Мы решили вторую проблему importance sampling'ом, но он не всегда поможет
- Например, источник света закрыт большим числом другой геометрии, или спрятан за сильно неоднородным стеклом (*frosted glass*)
- $\implies$ *Bidirectional path tracing (BDPT)*

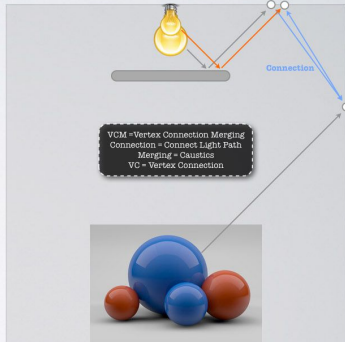
Bidirectional path tracing (Pixar RenderMan)

“Recessed Lighting Example”

Path Tracer: Uni Directional



VCM: Bi-Directional



Bidirectional path tracing (Veach, 1997)

- Возьмём area formulation уравнения рендеринга и переформулируем его как интеграл по *всем возможным путям*, которыми свет может пройти от источников света до камеры
- Нужно аккуратно формализовать пространство путей: возьмём объединение пространств всех путей конечной длины
- Нужно аккуратно формализовать меру на путях фиксированной длины N : она берётся как N -кратное произведение мер площади на поверхностях объектов
- Нужно аккуратно написать сам интеграл: в него будут входить значения BRDF промежуточных точек, а также *geometry terms*
- Нужно придумать, как семплить произвольные пути, и как считать их вероятности для Монте-Карло интегрирования

Path integral formulation

- Пусть мы сгенерировали путь $p_0 p_1 p_2 \dots p_N$
- Из точки p_0 в направлении точки p_1 излучилось $L_e(p_0 \rightarrow p_1)$ света
- Путь $p_0 \rightarrow p_1$ мог быть чем-то закрыт, за что отвечает *visibility term* $V(p_0 \rightarrow p_1)$
- Нужно учесть преобразование координат интегрирования из направлений на сфере в точки на поверхности:

$$\frac{\|\cos \theta_o(p_0) \cos \theta_i(p_1)\|}{\|p_1 - p_0\|^2}$$

- Вместе с $V(p_0 \rightarrow p_1)$ это даст *geometry term* $G(p_0 \rightarrow p_1)$
- Нужно учесть BRDF $f(p_0 \rightarrow p_1 \rightarrow p_2)$ в точке p_1 для света, пришедшего из p_0 и уходящего в p_2

- В итоге, количество света, полученного из этого пути, равно

$$\begin{aligned} &L_e(p_0 \rightarrow p_1) \cdot G(p_0 \rightarrow p_1) \cdot \\ &\cdot f(p_0 \rightarrow p_1 \rightarrow p_2) \cdot G(p_1 \rightarrow p_2) \cdot \\ &\cdot f(p_1 \rightarrow p_2 \rightarrow p_3) \cdot G(p_2 \rightarrow p_3) \cdot \\ &\quad \dots \\ &\cdot f(p_{N-2} \rightarrow p_{N-1} \rightarrow p_N) \cdot G(p_{N-1} \rightarrow p_N) \end{aligned}$$

Bidirectional path tracing

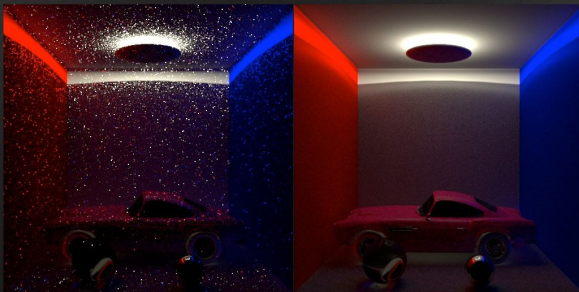
- Как генерировать случайные пути?
- Один из вариантов:
 - Сгенерировать случайный путь $p_0 \dots p_S$ из источника света
 - Сгенерировать случайный путь $q_0 \dots q_T$ из камеры
 - Склеить их в путь $p_0 \dots p_S q_T \dots q_0$
- На каждом шаге генерации пути останавливаемся с некоторой вероятностью p (тогда $\frac{1}{p}$ – средняя длина пути)
- Генерировать путь можно, пуская лучи из очередной точки в случайном направлении (можно с importance sampling'ом BRDF, например cosine-weighted для диффузных материалов)

Bidirectional path tracing

- Вместо того, чтобы обработать путь $p_0 \dots p_S q_T \dots q_0$ и выбросить, мы можем дополнительно попарно склеить все префиксы пути $p_0 \dots p_S$ со всеми префиксами $q_0 \dots q_T$ и получить квадратичное число путей
- BDPT позволяет улучшить сходимость Монте-Карло интегрирования, и позволяет обрабатывать гораздо более сложные сцены
- BDPT заметно сложнее в реализации
- **N.B.:** Существуют сцены, для которых задача решения уравнения рендеринга **алгоритмически неразрешима** (см. Reif et al. 1994)

Bidirectional path tracing (Pixar RenderMan)

Path vs VCM: "Recessed Lighting Example"



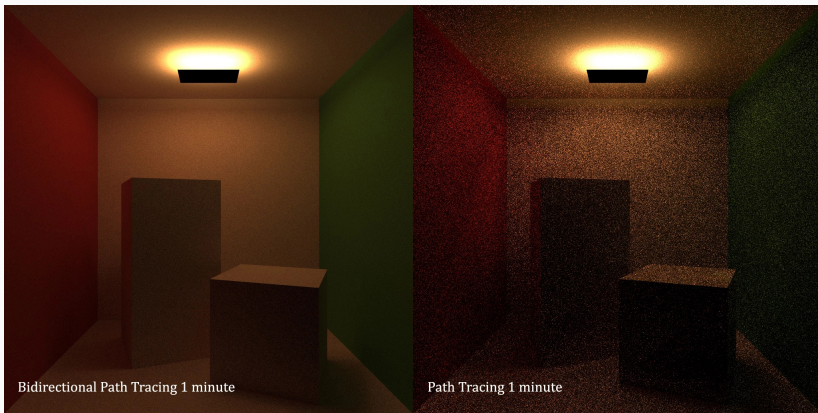
Integrator: Path
MaxPathLength=5
56sec

128 Max Samples

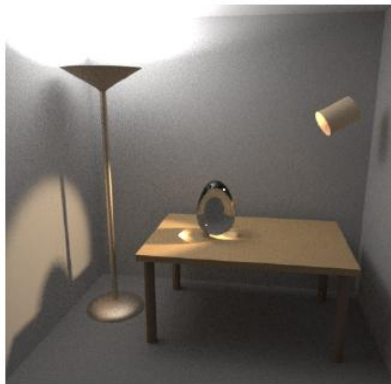
Integrator: VCM
MaxPathLength=5
4min

- Conclusion:
Uni-directional Path Tracer can't find path to light very well
Need bi-directional VCM in this case

Bidirectional path tracing (Sohil Shah)



Bidirectional path tracing (Sohil Shah)



(a) Bidirectional path tracing with 25 samples per pixel



(b) Standard path tracing with 56 samples per pixel (the same computation time as (a))

- Eric Veach PhD thesis, 1997 (chapters 8-10)
- Stanford raytracing course notes

Metropolis light transport

- Path integral formulation превращает задачу рендеринга в вычисление интеграла многомерной (вообще говоря, бесконечномерной) функции
- Для этой общей задачи известны хорошие методы, такие как алгоритм Метрополиса-Гастингса
- Комбинация этого алгоритма с path integral formulation даёт подход, известный как *metropolis light transport*

Алгоритм Метрополиса-Гастингса

- Пусть, есть распределение вероятности $p(x)$ (на каком-то множестве), которое мы знаем с точностью до постоянного множителя (т.е. не знаем нормировку)
- Другими словами, мы знаем некоторую функцию $f(x) = C \cdot p(x)$, но не знаем C
- Мы хотим генерировать случайные семплы X с вероятностью $p(x)$

Алгоритм Метрополиса-Гастингса

- Выберем какой-нибудь первый семпл X_0
- Выберем некоторое распределение $g(x \rightarrow y)$ (*proposal function*), которое по текущей точке x будет генерировать следующую точку y
- Proposal function должна быть симметричной:
 $g(x \rightarrow y) = g(y \rightarrow x)$
- Далее в цикле:
 - Генерируем новый семпл Y используя proposal распределение $g(X_i \rightarrow Y)$
 - Вычисляем *acceptance ratio* $\alpha = \frac{f(Y)}{f(X_i)}$
 - С вероятностью $\min(\alpha, 1)$ положим $X_{i+1} = Y$, иначе $X_{i+1} = X_i$
- Тогда последовательность семплов $\{X_i\}$ в пределе стремится к распределению $p(x)$

Алгоритм Метрополиса-Гастингса

- Если $f(Y) > f(X_i)$, то $\alpha > 1$, и новый семпл будет всегда принят $X_{i+1} = Y$
- Чем меньше отношение $\alpha = \frac{f(Y)}{f(X_i)}$, тем меньше вероятность, что семпл будет принят
- Интуитивно, алгоритм идёт в направлении увеличения веса семплов, иногда заходя в регионы с меньшей вероятностью
- Если случайная величина задана на $\mathbb{R}^n$, в качестве $g(x \rightarrow y)$ часто берут Гауссово распределение $g(x \rightarrow y) \sim \exp(-\|x - y\|^2)$

- Алгоритм очень чувствителен к выбору первого семпла X_0 (*start-up bias*), поэтому часто используют *burn-in period*: большое количество первых семплов выкидывают
- Последовательные семплы алгоритма скоррелированы, что может негативно влиять на результат вычислений

Метрополис-Гастингс + Монте-Карло интегрирование

- Хотим вычислить интеграл $I = \int f(x)dx$ от некоторой функции $f(x)$
- Выбираем плотность вероятности $p(x)$, похожую на нашу функцию, которую мы умеем нормировать
- Используем алгоритм Метрополиса-Гастингса с плотностью $p(x)$
- Делаем burn-in, вхолостую генерируя набор семплов
- После чего генерируем N семплов
- Аппроксимируем интеграл как

$$I \approx \frac{1}{N} \sum \frac{f(X_i)}{p(X_i)}$$

Метрополис-Гастингс + Монте-Карло + importance sampling

- Хотим вычислить интеграл $I = \int f(x)g(x)dx$ от произведения некоторых функций
- Выбираем плотность вероятности $p(x) \sim f(x)$, пропорциональную $f(x)$

$$p(x) = \frac{f(x)}{\int f(y)dy}$$

- Используем алгоритм Метрополиса-Гастингса с плотностью $f(x)$
- Делаем burn-in, вхолостую генерируя набор семплов
- Генерируем N семплов
- Аппроксимируем интеграл как

$$I \approx \frac{1}{N} \sum \frac{f(X_i)g(X_i)}{p(X_i)} = \left[\frac{1}{N} \sum g(X_i) \right] \cdot \left[\int f(x)dx \right]$$

Metropolis light transport

- Нужно придумать, какую использовать плотность вероятности (возможно, пропорциональную какому-то множителю), и учесть то, что мы не знаем нормировки (возможно, её легче почитать через BDPT)
- Нужно сгенерировать первый путь (семпл), например, так же, как в BDPT
- Нужно придумать proposal distribution – случайные мутации пути
- Veach, Guibas - Metropolis Light Transport
- PBR Book: Metropolis Light Transport

- Один из самых распространённых сегодня форматов описания трёхмерных сцен
- Поддерживает иерархии объектов и их преобразований
- Поддерживает текстуры и сложные материалы
- Поддерживает анимированные объекты
- Поддерживает источники света (в виде расширения)
- Удобен для загрузки на GPU для рендеринга (например, через OpenGL или Vulkan)

- Особенность формата в том, что он хранит метаданные (описание дерева объектов, и т.д.) в текстовом виде (в JSON'е), а бинарные данные (координаты отдельных вершин, индексы вершин, keyframe'ы анимаций) – в сыром бинарном виде
- Бинарные данные обычно представлены ссылкой на внешний источник (файл, URL, etc)
- Есть вариация формата (GLB), которая склеивает JSON и бинарные данные в один файл
- Ссылка на спецификацию формата

Формат glTF: структура

- Объект формата glTF – это JSON-объект, большинство полей которого это массивы объектов определённого *типа*: nodes, meshes, accessors, buffers, bufferViews, materials, cameras, ...
- Объекты ссылаются друг на друга с помощью индексов в эти массивы
- Ноды образуют виртуальное дерево объектов сцены (необязательно видимых объектов), и могут содержать другие ноды в качестве детей
- Каждая нода может иметь матрицу преобразования относительно родительской ноды (или относительно центра сцены, если это корневая нода)

- Нода может иметь меш (тогда эта нода подлежит рисованию)
- Меш содержит несколько примитивов, каждый из которых имеет свой материал и набор вершин
- Позиции вершин и индексы вершин в треугольниках описываются *аксессорами*
- Аксессоры задают тип и формат данных, и ссылаются на `buffer view`, которые ссылаются на буфер с бинарными данными

- Нода может содержать камеру – тогда позиция и ориентация этой ноды задают позицию и ориентацию камеры
- Вид и параметры проекции камеры задаются соответствующим объектом из массива `cameras`

- Для парсинга формата нужен какой-нибудь парсер JSON
- C++: Rapidjson, nlohmann/json
- Rust: json-rust

- Для использования в нашем рейтрейсере, нам нужно превратить glTF-сцену в
 - Набор вершин
 - Набор треугольников
 - Набор материалов
 - Камеру

- В качестве камеры берём первую попавшуюся ноду с камерой (считаем, что такая нода ровно одна)
- Поддерживаем только перспективную камеру
- Позицию и ориентацию камеры берём из содержащей её ноды

- Треугольник описывается тройкой индексов вершин и индексом материала
- Можно хранить разные объекты (ноды/меш/примитивы) отдельно, а можно слить всё в одну кучу треугольников
- В любом случае, нужно учесть преобразование ноды – его можно применить к вершинам сразу, а можно применять обратное преобразование непосредственно при поиске пересечения
- Можно построить один BVH на все треугольники, а можно разбить на отдельные TLAS + BLAS

- Преобразования нод могут быть заданы или явной матрицей 4×4 (в однородных координатах), или тройкой translation-rotation-scale
- Translation задаётся трёхмерным вектором
- Rotation задаётся единичным кватернионом
- Scale задаётся трёхмерным вектором и может описывать неизотропное масштабирование (*nonuniform scaling*) – его нужно покомпонентно умножать на координаты вершины
- К преобразованию ноды нужно рекурсивно добавить преобразования всех родительских нод

- От самих вершин нам пока нужны только координаты (не забываем к ним применить преобразование ноды)
- Они могут быть заданы в большом количестве разных форматов; будем считать, что они всегда в виде трёхмерных 32-битных floating point
- Также будем считать, что индексы или 16-битные, или 32-битные беззнаковые целые числа (придётся поддерживать два варианта)
- Обратите внимание, что `byteOffset` есть и в аксессуарах, и в `bufferView`
- Параметр `byteStride` в `bufferView` будем игнорировать

Формат glTF: реализация

- glTF поддерживает довольно сложные материалы; пока мы будем использовать только несколько параметров
- `pbrMetallicRoughness.baseColorFactor` – цвет материала (RGBA)
- Если альфа-канал цвета меньше 1, считаем материал диэлектриком с $IOR = 1.5$
- Если `pbrMetallicRoughness.metallicFactor` больше 0, считаем объект металлом
- Иначе объект диффузный
- В любом случае, `emissiveFactor` – интенсивность излучения материала

- Обратите внимание на многочисленные дефолты: например, `baseColorFactor` по умолчанию равен `[1, 1, 1, 1]`, а `metallicFactor` равен 1
- По спецификации, `emissiveFactor` не может быть больше 1; это обходится расширением `KHR_materials_emissive_strength` (см. примеры сцен для практики)